MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova Faculty of Computers, Informatics and Microelectronics Department of Software and Automation Engineering

Lupasco Cristian FAF-233

Report

Laboratory work n.4

of Languages and Finite Automata

Checked by:
Cretu D., DISA, FCIM, UTM

Chișinău – 2024

Theory

Regular expressions, commonly referred to as regex, are symbolic notations used to define search patterns within strings. They are widely utilized in fields such as text processing, lexical analysis, and data validation. A regular expression consists of symbols and operators that describe a set of strings, and is rooted in formal language theory—specifically in regular languages defined by finite automata.

In the context of this laboratory work, we explored how simplified regex patterns can be interpreted and dynamically expanded into valid string combinations. Rather than using built-in regex matchers for checking if a string matches a pattern, the task focused on simulating the construction of such strings from the pattern itself. This mimics the process of recognizing and producing words from a regular language, similar to how finite automata or generators work.

To achieve this, the implementation involved parsing components like repetitions (*, +), optional symbols (?), group choices ((a—b)), and exponentiation ((a—b)3). Each symbol or structure carries specific semantics which were respected during the expansion. The challenge was to model these rules using Python logic while incorporating randomness to reflect the variability allowed by regular expressions. This hands-on approach solidified our understanding of regex operations and demonstrated how theoretical constructs can be transformed into algorithmic solutions.

Conditions of the Problem

1. Write and cover what regular expressions are, what they are used for;
2. Write a code that will generate valid combinations of symbols conform given regular expressions (examples will be shown). Be careful that idea is to interpret the given regular expressions dinamycally, not to hardcode the way it will generate valid strings. You give a set of regexes as input and get valid word as an output;
3. In case you have an example, where symbol may be written undefined number of times, take a limit of 5 times (to evade generation of extremely long combinations);
4. Bonus point: write a function that will show sequence of processing regular expression (like, what you do first, second and so on);

Technical Implementation

```
def generate_strings_from_regex(pattern, limit=5):
    steps = []
    ...
    return pattern, steps
```

Purpose: This is the main function that takes a simplified regex pattern and generates a string that matches it, using random values to simulate repetitions and choices. It also logs each transformation step into a list for transparency. It acts as the engine of the string generation process and calls other helper functions using regular expression substitutions to progressively expand the pattern into a valid string.

```
   def expand_exponentiation(match):
    choices = match.group(1).split('|')
    count = int(match.group(2))
    selection = ''.join(random.choice(choices) for _ in
       range(count))
    steps.append(f"Expanding␣exponentiation␣{match.group
       (0)}␣->␣{selection}")
    return selection
```

Purpose: Expands patterns like (a—b)3 by choosing a random element from the group (a or b) three times. It uses group(1) to extract the characters and group(2) for the repetition count. This simulates custom exponentiation not natively supported in regular expressions and allows pattern flexibility in generating controlled-length strings.

```
    def expand_repetitions(match):
    char = match.group(1)
    op = match.group(2)
    count = random.randint(1, limit) if op == '+' else
       random.randint(0, limit)
    result = char * count
    steps.append(f"Expanding␣repetition␣{match.group(0)}␣
       ->␣{result}")
    return result
```

Purpose: Expands basic repetition operators like a+ and a*. The function randomly decides how many times to repeat a character, ensuring at least once for + and possibly zero for *. This mimics the Kleene plus and star operations from formal language theory and supports flexible pattern generation.

```
  def expand_group(match):
    group_content = match.group(1)
    repetition = match.group(2)
    choices = group_content.split('|')
    count = random.randint(1, limit) if repetition == '+'
       else random.randint(0, limit)
    selection = ''.join(random.choice(choices) for _ in
       range(count))
    steps.append(f"Expanding␣group␣{match.group(0)}␣->␣{
       selection}")
    return selection
```

Purpose: Handles grouped repetitions like (x—y—z)+ or (a—b)*, randomly selecting and repeating items from a list. This simulates complex group repetitions, similar to how NFA or DFA might handle loops over sets of inputs, and gives variety to the generated strings while respecting the structure.

```
def expand_simple_group(match):
    choices = match.group(1).split('|')
    selection = random.choice(choices)
    steps.append(f"Selecting␣from␣group␣{match.group(0)}␣
        ->␣{selection}")
    return selection
```

Purpose: Handles plain groups like (a—b—c) without repetitions. It selects a single element randomly. This is key for interpreting alternation in regex, and helps implement non-deterministic decisions in pattern paths.

```
    def expand_optional(match):
    selection = match.group(1) if random.choice([True,
        False]) else ''
    steps.append(f"Expanding␣optional␣{match.group(0)}␣->␣
        {selection}")
    return selection
```

Purpose: Handles optional elements such as a?, randomly deciding whether to include the character. It simulates epsilon-transitions in automata by optionally removing a symbol from the pattern path.

Visual Representation

```
Regex: J+K(L|M|N)*O?(P|Q)^3
Generated: JJJKLNNMOQQQ
Processing Steps:
  - Expanding exponentiation (P|Q)^3 -> QQQ
  - Expanding group (L|M|N)* -> LNNM
  - Expanding repetition J+ -> JJJ
  - Expanding optional O? -> O

Generated: JJKNLLMOQPQ
Processing Steps:
  - Expanding exponentiation (P|Q)^3 -> QPQ
  - Expanding group (L|M|N)* -> NLLM
  - Expanding repetition J+ -> JJ
  - Expanding optional O? -> O

Generated: JJJJJKOQQQ
Processing Steps:
  - Expanding exponentiation (P|Q)^3 -> QQQ
  - Expanding group (L|M|N)* ->
  - Expanding repetition J+ -> JJJJJ
  - Expanding optional O? -> O

Generated: JJKMOQPQ
Processing Steps:
  - Expanding exponentiation (P|Q)^3 -> QPQ
  - Expanding group (L|M|N)* -> M
  - Expanding repetition J+ -> JJ
  - Expanding optional O? -> O

Generated: JJKLNMMPPP
Processing Steps:
  - Expanding exponentiation (P|Q)^3 -> PPP
  - Expanding group (L|M|N)* -> LNMM
  - Expanding repetition J+ -> JJ
  - Expanding optional O? ->
```

Figure 1: Screenshot from PyCharm

Conclusions

The regex string generation script demonstrates a modular and intuitive approach to expanding simplified regular expression patterns into concrete string examples. Each helper function is tailored to interpret and simulate specific regex components, such as optional symbols, repetition (*, +), group choices, and custom exponentiation like (a—b)3. Through systematic use of regular expression matching and replacement, the program transforms abstract patterns into real-world string outputs while maintaining a clear log of each step in the expansion process.

This method not only aids in visualizing how regex components work but also serves as an effective tool for debugging, teaching, or testing regex logic. The inclusion of randomness introduces variation, reflecting the non-deterministic nature of many regular expressions. Moreover, separating concerns into specialized functions improves readability, scalability, and maintainability of the code.

In summary, this script bridges the gap between theoretical regex structures and practical examples, providing insight into how expressions are interpreted and instantiated. It's a valuable tool for students, developers, and testers seeking a clearer understanding of regex behavior in controlled environments.